LAB 6 : From Linear Estimation to Learning Shapes

Why Activations, Slopes, and Backward Flow Become Necessary

1) What "Learning" Has Meant So Far (Recap)

Until now, every model you built had this structure:

Perceptron (classification)

$$z = w^\top x + b$$

$$\hat{y} = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

Learning idea (conceptual):

- inputs x stay fixed
- prediction ŷ is produced by the same rule
- only the numbers w and b change when the model is wrong

You saw a simple update rule:

$$w := w + \eta \cdot (y - \hat{y}) \cdot x$$

$$b := b + \eta \cdot (y - \hat{y})$$

The symbol η (eta) is the learning rate. It controls how big each update step is.

This worked because logic gates are simple and separable.

Linear neuron (regression)

Same core structure, but output is numeric:

$$\hat{y} = w^\top x + b$$

Now the model output is not a label. It is an estimate.

So you needed a way to measure “how wrong” the estimate is.

A common error signal:

$$e = \hat{y} - y$$

A common loss (MSE for one sample):

$$L = (\hat{y} - y)^2$$

Learning still means:

keep the same equation form

change w and b to reduce loss

2) The New Problem: The Shape Can Be Wrong

What “shape” means

A model is a function.

The equation you choose defines what kinds of patterns your model can represent.

For a single input x, the model:

$$\hat{y} = wx + b$$

is always a straight line.

For multiple inputs x∈Rd

$$\hat{y} = w^\top x + b$$

is always a flat surface (a hyperplane).

A hyperplane is the higher-dimensional version of a straight line (2D) or a plane (3D).

No matter how you change w and b, this function class stays the same:

- straight line in 1D
- flat surface in higher dimensions

What “bending” means (important)

Bending” means the model output changes at different rates in different regions.

A straight line has one constant slope everywhere.

A bent function has different slopes depending on the input region.

Examples where bending is needed:

- growth slows after maturity
- salary increases quickly early in career, slower later
- age vs size increases fast at first then saturates

A linear model cannot represent “fast then slow” without systematic error.

This produces repeated bias like:

- underestimating young cases
- overestimating older cases

even after tuning weights

This is not a tuning issue.

This is a representation issue.

3) Why We Need “Direction” (Slope) to Learn

If the model is wrong, we must change parameters w, b.

But the key question becomes:

Which direction should we change them?

Changing a parameter can:

- decrease loss
- increase loss

So learning needs a direction signal.

Slope (intuitive definition)

Slope is a number that tells:

If I increase a parameter slightly, what happens to the loss?

Using a small change idea:

$$\text{slope} \approx \frac{\Delta L}{\Delta w}$$

Interpretation:

- if slope is positive, increasing w increases loss → move w down
- if slope is negative, increasing w decreases loss → move w up

So the general “move opposite slope” rule is:

$$w := w - \eta \cdot \text{slope}$$

This is gradient descent logic in plain form.

4) Slope vs Gradient (Difference)

Slope

Slope is for one parameter:

$$\frac{\partial L}{\partial w}$$

It is “how loss changes with one number”.

Gradient

Gradient is the collection of slopes for all parameters.

If parameters are a vector:

$$w = (w_1, w_2, \ldots, w_d)$$

then:

$$\nabla_w L = \left(\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \ldots, \frac{\partial L}{\partial w_d} \right)$$

So:

- slope = one direction for one parameter
- gradient = direction information for all parameters together

Update rule becomes:

$$w := w - \eta \cdot \nabla_w L$$

5) Activation Function: What It Is and Why It Exists

The core limitation

If you stack linear steps, you still get a linear function.

Example (one input):

$$z = w_1 x + b_1$$

$$\hat{y} = w_2 z + b_2$$

Substitute:

$$\hat{y} = w_2(w_1 x + b_1) + b_2$$

$$\hat{y} = (w_2 w_1)x + (w_2 b_1 + b_2)$$

This is still:

$$\hat{y} = \tilde{w} x + \tilde{b}$$

So multiple linear layers without activation do not bend anything.

They collapse into one linear layer.

Activation function definition

An activation function is a non-linear function applied to a score z:

$$h = \phi(z)$$

It changes the shape of what the model can represent.

After activation, the model can become “piecewise” or “smoothly curved” instead of always flat.

Activation vs Loss vs Gradient (must be clear)

Activation

- belongs to the model architecture
- changes how signals transform
- changes representational power

$$h = \phi(z)$$

Loss

- belongs to evaluation
- measures how wrong predictions are

$$L = (\hat{y} - y)^2$$

or for classification:

$$L_{CE} = -\left(y \log(\hat{p}) + (1 - y) \log(1 - \hat{p})\right)$$

Gradient

- belongs to learning procedure
- tells how to adjust parameters to reduce loss

$$\nabla_\theta L$$

So:

- activation = bends the function
- loss = scores the mistake
- gradient = gives direction to reduce mistake

6) The Two Activations We Use First

ReLU

$$\mathrm{ReLU}(z) = \max(0, z)$$

Meaning:

- negative input becomes 0 (inactive)
- positive input passes through (active)

Derivative used in learning:

$$\mathrm{ReLU}'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Meaning:

- maps any real number to (0,1)
- smooth transition from 0 to 1

Derivative used in learning:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

7) What Changes From Perceptron Thinking

Old structure (perceptron / linear neuron)

$$\hat{y} = w^\top x + b$$

Learning directly updates w, b.

New structure (multi-step model)

$$z = W_1^\top x + b_1$$

$$h = \phi(z)$$

$$\hat{y} = W_2^\top h + b_2$$

Now parameters are:

- first layer parameters: W_1, b_1
- second layer parameters: W_2, b_2

The learning target stays the same: reduce loss

But now:

$$W_1 \to z \to h \to \hat{y} \to L$$

Learning must compute how early parameters affect loss through later steps.

That computation is backward flow.

8) Forward Pass and Backward Pass

Forward pass:

$$x \to z \to h \to \hat{y}$$

Produces prediction and stores intermediates.

Backward pass:

$$L \to \hat{y} \to h \to z \to (W_1, b_1, W_2, b_2)$$

This is structured multiplication of local slopes.

9) “Backward Flow” as a Chain of Slopes

Loss:

$$L = (\hat{y} - y)^2$$

Error:

$$e = \hat{y} - y$$

$$\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y)$$

If:

$$\hat{y} = W_2^\top h + b_2$$

Local slopes:

$$\frac{\partial \hat{y}}{\partial W_2} = h$$

$$\frac{\partial \hat{y}}{\partial b_2} = 1$$

$$\frac{\partial \hat{y}}{\partial h} = W_2$$

If:

$$h = \phi(z)$$

$$\frac{\partial h}{\partial z} = \phi'(z)$$

If:

$$z = W_1^\top x + b_1$$

$$\frac{\partial z}{\partial W_1} = x$$

$$\frac{\partial z}{\partial b_1} = 1$$

Chain rule gradients:

$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial W_2}$$

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h} \cdot \frac{\partial h}{\partial z} \cdot \frac{\partial z}{\partial W_1}$$

This multiplication chain is the chain rule in working form.

10) Parameter Update (Gradient Descent Form)

For any parameter θ:

$$\theta := \theta - \eta \cdot \frac{\partial L}{\partial \theta}$$

$$W_1 := W_1 - \eta \cdot \frac{\partial L}{\partial W_1}$$

$$b_1 := b_1 - \eta \cdot \frac{\partial L}{\partial b_1}$$

$$W_2 := W_2 - \eta \cdot \frac{\partial L}{\partial W_2}$$

$$b_2 := b_2 - \eta \cdot \frac{\partial L}{\partial b_2}$$

This is exactly the same idea as perceptron updates:

- change numbers to reduce error

The difference is:

- we now must compute gradients through multiple steps
- so we need forward pass (store intermediates) and backward pass (compute gradients)